CSE493L Section 10

Misc Topics and Final Review

Calling C from Rust

C Function

What preconditions must be satisfied to avoid undefined behavior?

```c
// PRECONDITIONS:
// - ptr is non-null and aligned.
// - ptr points to at least `len` writable `int` values.
// - we have exclusive access to this memory

void c_increment_array(int* ptr, size_t len) {
   for (size_t i = 0; i < len; i++) {
       ptr[i]++;
   }
}
```

```rust
// Linking with Rust:
use std::ffi::c_int;
#[link(name = "my_c_library")]
unsafe extern "C" {
   fn c_increment_array(ptr: *mut c_int, len: usize);
}
```

Calling It From Rust

- Which precondition was accidentally violated here?
- Manually checking preconditions each time is hard.
- How can we avoid this?
  - Create a safe wrapper!

```rust
fn main() {
    let mut arr = [0, 0, 0, 0];
    let slice: &mut [i32] = &mut arr[0..2];

    unsafe {
        c_increment_array(slice.as_mut_ptr(), 200);
    }
    println!("{slice:?}");
}
```

Safe Wrapper

Why is this better than just always calling the C function directly?

Lets us rely on “soundness”!

No matter how a safe function is called from safe code, it won’t cause undefined behavior.

```rust
fn safe_increment_slice(slice: &mut [i32]) {
    // GUARANTEED for &mut [i32]:
    // - ptr is non-null and aligned.
    // - ptr points to `len` writable `int` values.
    // - we have exclusive access to this memory
    unsafe {
        c_increment_array(slice.as_mut_ptr(), slice.len());
    }
}

fn main() {
    let mut arr = [0, 0, 0, 0];
    let slice: &mut [i32] = &mut arr[0..2];

    safe_increment_slice(slice);
    println!("{slice:?}");
}
```

Burden of Proof

Scenario:	**Burden of proof falls on:**
Unsafe function	Function caller (on each call)
Safe function that wraps unsafe	Function author (only once)
Safe function with only safe Rust	Compiler (automatic)

Can We Automatically Prove More Preconditions?

Yes, with a formal deductive verifier like Verus.

Simple demo: before, after

- Full formal verification is very tedious.
  - Use only for high-stakes situations.
  - For example, a confidential virtual machine security module.
- Write tests for everything else!
  - **cargo llvm-cov** to check coverage.
  - **loom** to explore all interleavings.

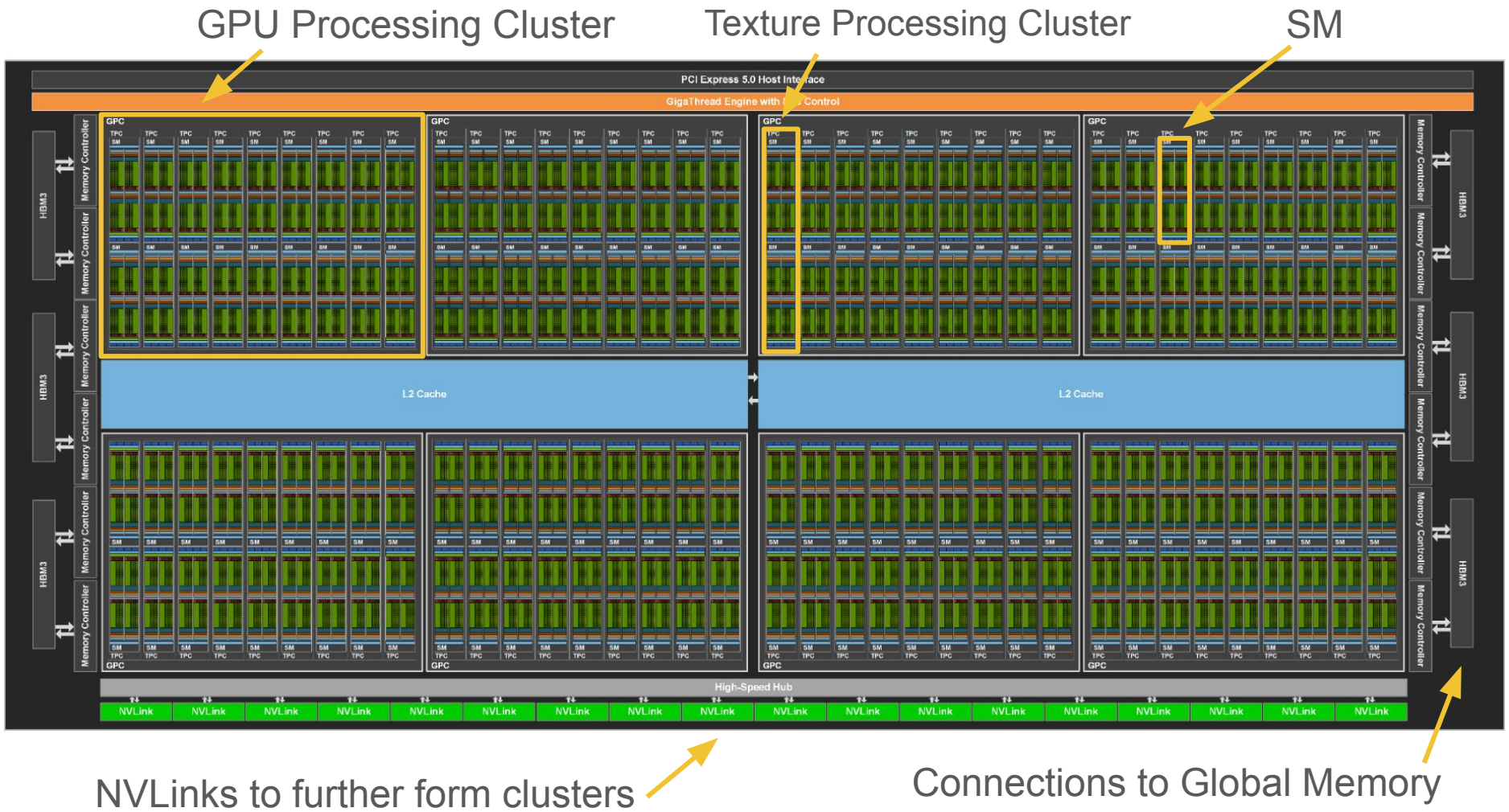
GPUs

GPU Memory Hierarchy

NVIDIA GH100 PCIe Gen 5 is made of 144 Streaming Multiprocessors (SMs)

- 128 FP32 Cores / SM
- 64 Warps Max / SM
  - 32 Threads / Warp
- 4 Tensor Cores / SM
- 65,536 Registers / SM
  - 255 Registers Max / Thread
- Up to 228KB Shared Memory (scratchpad)
  - Combined with L1 Cache, for 256KB total
- 60MB L2 Cache

… and more. These things are huge, this list is incomplete.


GPU Processing Cluster
Texture Processing Cluster
SM
PCI Express 5.0 Host Interface
GigaThread Engine with MIG Control
GPC
TPC
SM
L2 Cache
L2 Cache
Memory Controller
HBM3
High-Speed Hub
NVLink
NVLinks to further form clusters
Connections to Global Memory

GPU Memory Hierarchy

SMs are further split into SM Sub-Partitions

- This is what finally schedules warps!

To actually execute instructions:

1. Programmer defines “blocks”
   a. Groups of SMs which operate together to accomplish some task
2. Compiler/Hardware splits blocks into warps
   a. 32 threads that will operate together in lockstep
3. Warps are dynamically scheduled within an SM, never moving between them

GPU Memory Hierarchy

"Warps are dynamically scheduled within an SM"

- If a block spans many warps, then synchronization and coordination is required
- Between SMSPs → L1 Cache can be used
- Between SMs → L2 Cache can be used

That's why the scratchpad memory is so useful!

- Programmer managed, as fast as L1 cache
- Accessed by individual SMSPs, which are together executing a single "thread block"

Using Shared Memory

*which is still a huge simplification, there's more efficient ways of doing this

Consider this program*, which just reverses a 64-element array of elements:

```
int main(void)
{
  const int n = 64; int a[n], r[n];
  // initialize input array a, and expected output array r
  ...
  int *d_d;
  cudaMalloc(&d_d, n * sizeof(int));

  cudaMemcpy(d_d, a, n*sizeof(int), cudaMemcpyHostToDevice);
  staticReverse<<<1,n>>>(d_d, n);
  cudaMemcpy(d, d_d, n*sizeof(int), cudaMemcpyDeviceToHost);
  for (int i = 0; i < n; i++)
    if (d[i] != r[i]) printf("Error: d[%d]!=r[%d] (%d, %d)n", i, i, d[i], r[i]);
}
```

64 elements will require at least 2 warps!

Using Shared Memory

*which is still a huge simplification, there's more efficient ways of doing this

Consider this program*, which just reverses a 64-element array of elements:

```
int main(void)
{
  const int n = 64; int a[n], r[n];
  // initialize input array a, and expected output array r
  ...
  int *d_d;
  cudaMalloc(&d_d, n * sizeof(int));

  cudaMemcpy(d_d, a, n*sizeof(int), cudaMemcpyHostToDevice);
  staticReverse<<<1,n>>>(d_d, n);
  cudaMemcpy(d, d_d, n*sizeof(int),
  for (int i = 0; i < n; i++)
    if (d[i] != r[i]) printf("Error: d[%d]!=r[%d] (%d, %d)n", i, i, d[i], r[i]);
}
```

64 elements will require at least 2 warps!

Operation uses 1 block (on 1 SM), with `n` threads / block

Using Shared Memory

Consider this program, which just reverses a 64-element array of elements:

1. Declare 64 element array in shared memory
2. Each thread moves one element into shared memory, then waits for all threads
3. Each thread moves one element into destination, in reverse order

```
__global__ void staticReverse(int *d, int n)
{
  __shared__ int s[64];
  int t = threadIdx.x;
  int tr = n-t-1;
  s[t] = d[t];
  __syncthreads();
  d[t] = s[tr];
}
```

Using Shared Memory

Consider this program, which just reverses a 64-element array of elements:

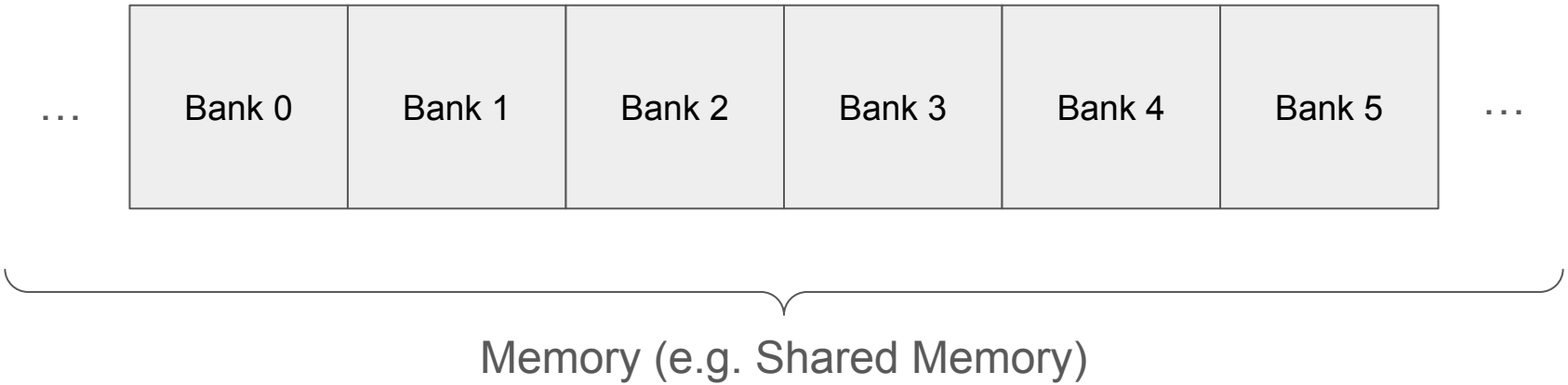
1. Declare 64 element array in shared memory
2. Each thread moves one element into shared memory, then waits for all threads
3. Each thread moves one element into destination, in reverse order

```
__global__ void staticReverse(int *d, int n)
{
  __shared__ int s[64];
  int t = threadIdx.x;
  int tr = n-t-1;
  s[t] = d[t];
  __syncthreads();
  d[t] = s[tr];
}
```

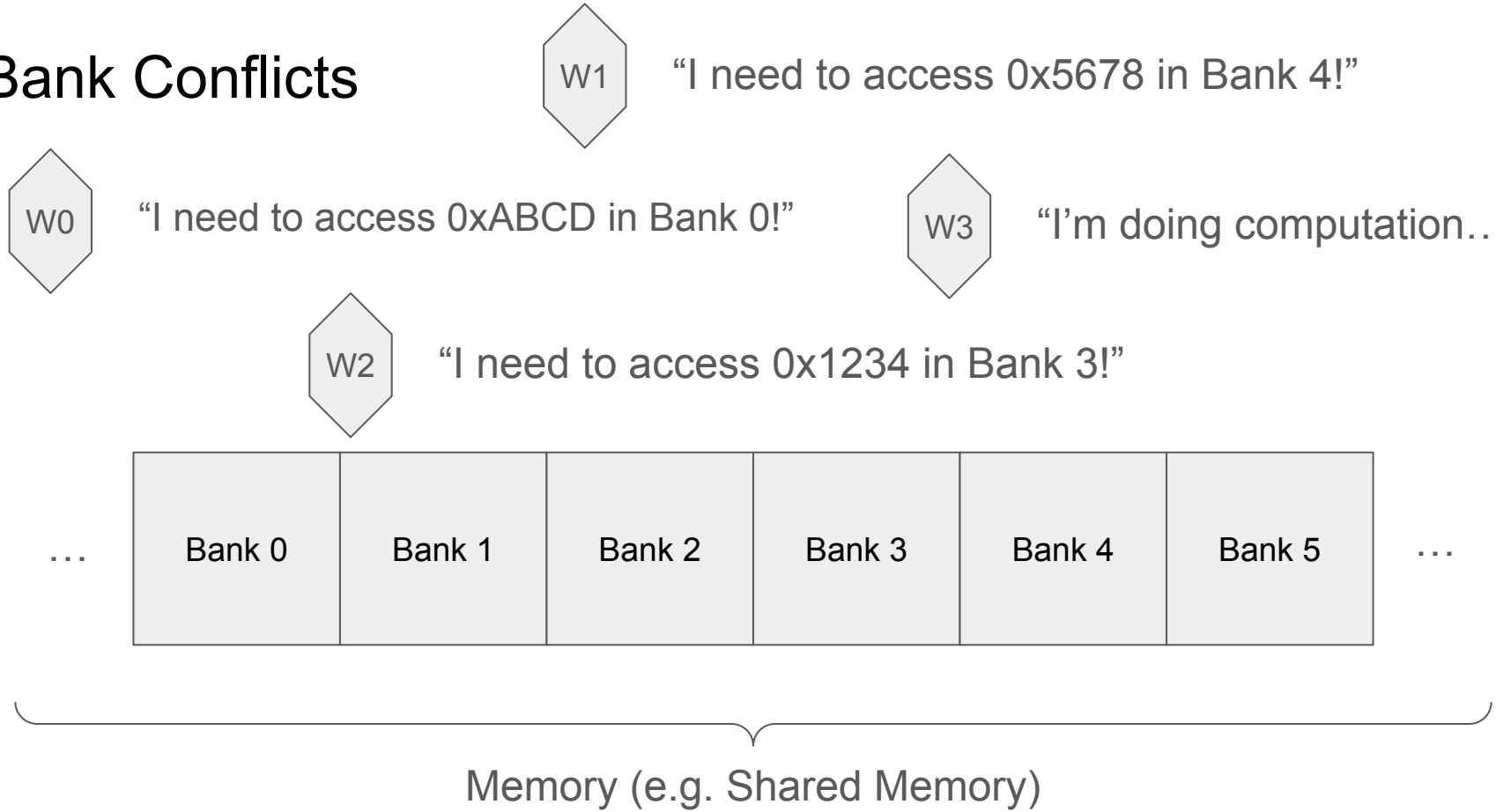
Why do we need the barrier?

Bank Conflicts

All memory is physically split into “banks” that allow for parallel access.

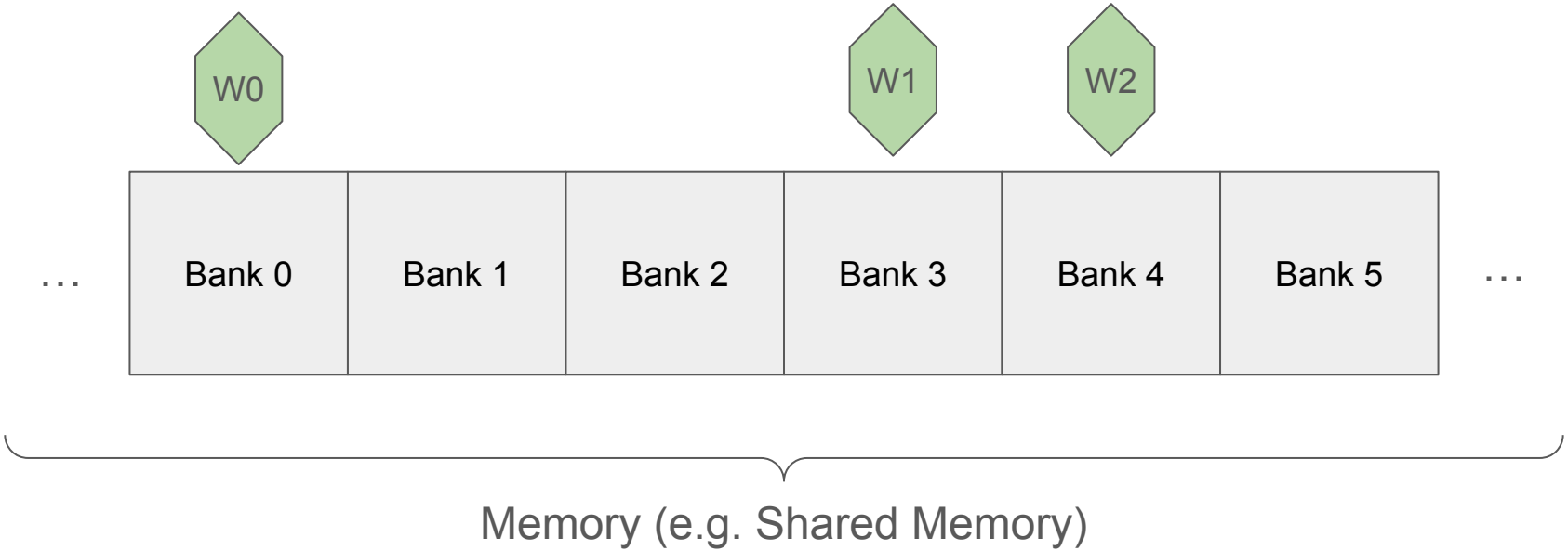


Bank Conflicts



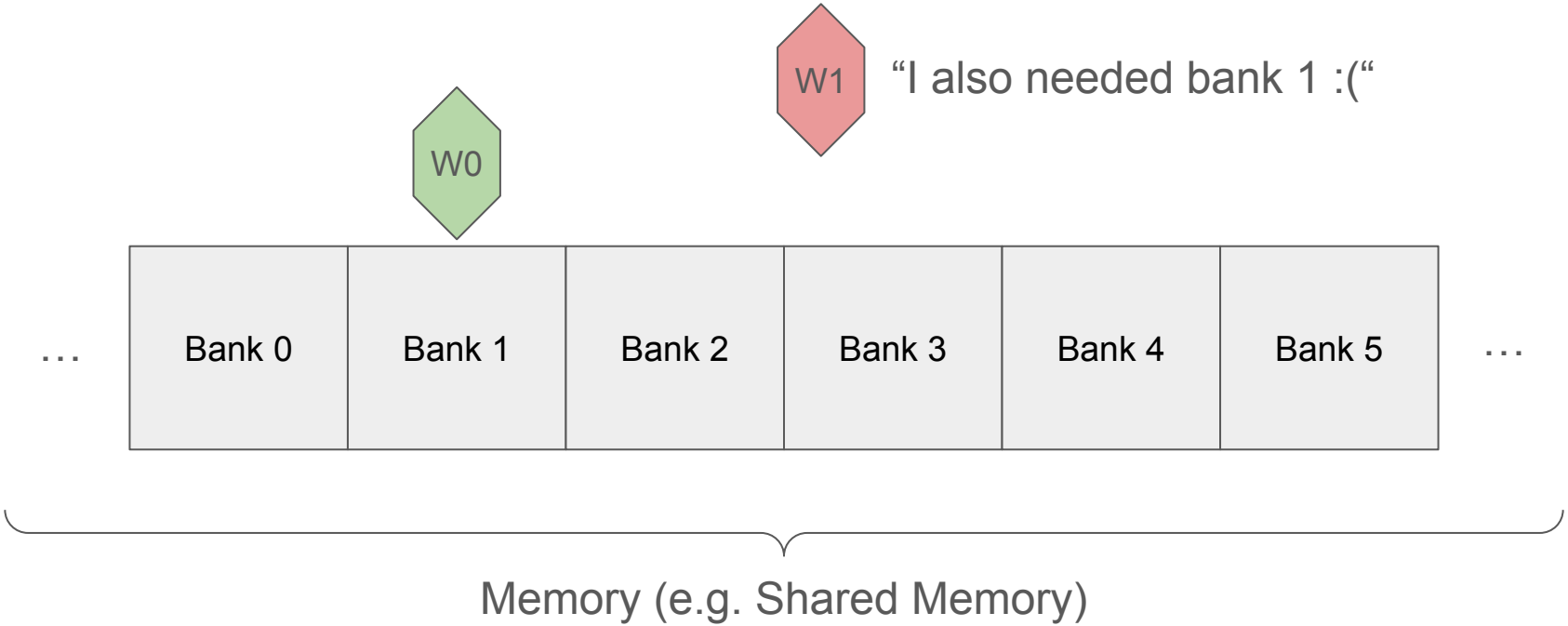
Bank Conflicts

All of the memory accesses can be done in parallel with one another!



Bank Conflicts

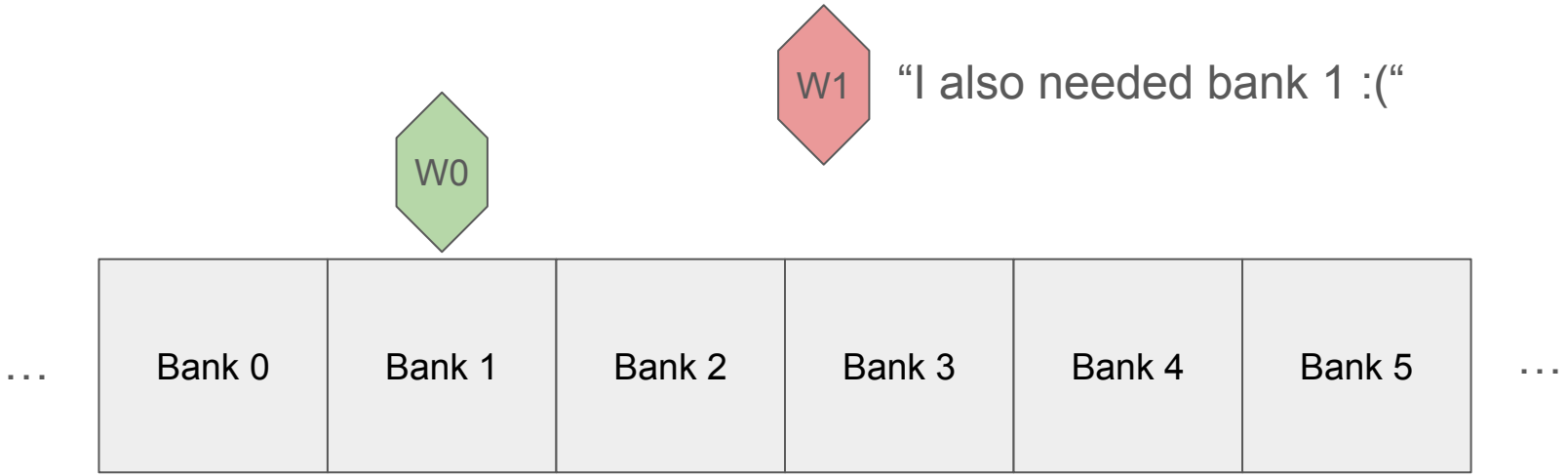
If two warps need to access the same bank, one of them will have to wait.



Bank Conflicts

If two warps need to access the same bank, one of them will have to wait.

- This can cause significant slowdowns… profile your code to find where it happens, add padding as necessary to move data into different banks.

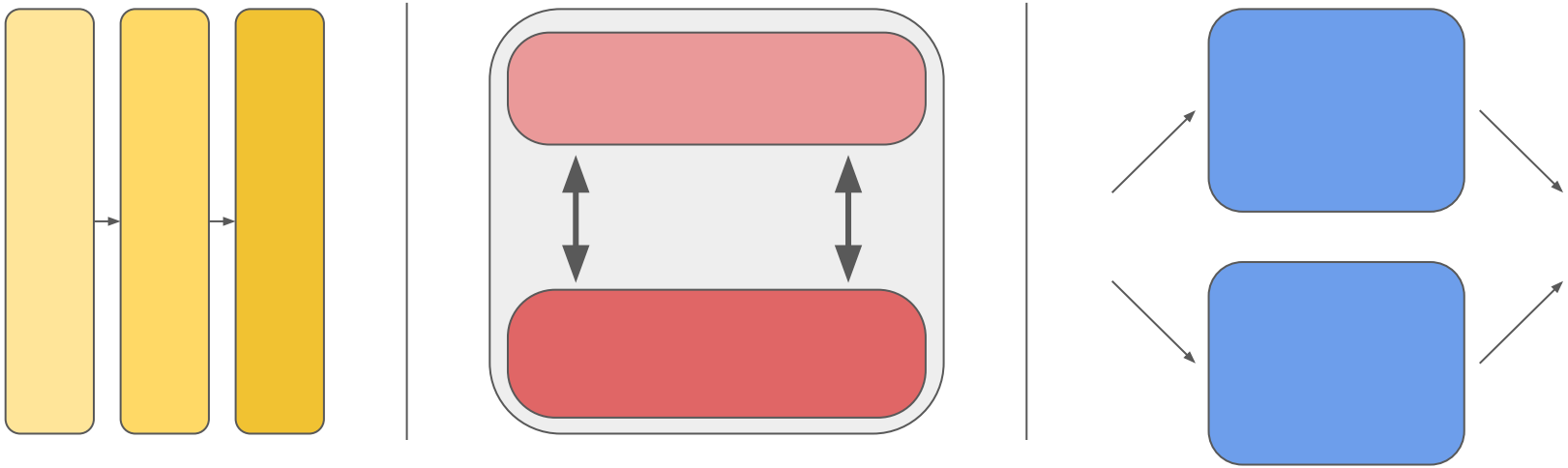


Machine Learning Parallelism

Parallelism in Machine Learning

Llama 3 was trained over two clusters, with ~24,576 GPUs each

- Requires a crazy amount of effort to split and coordinate work effectively



Pipeline Parallelism

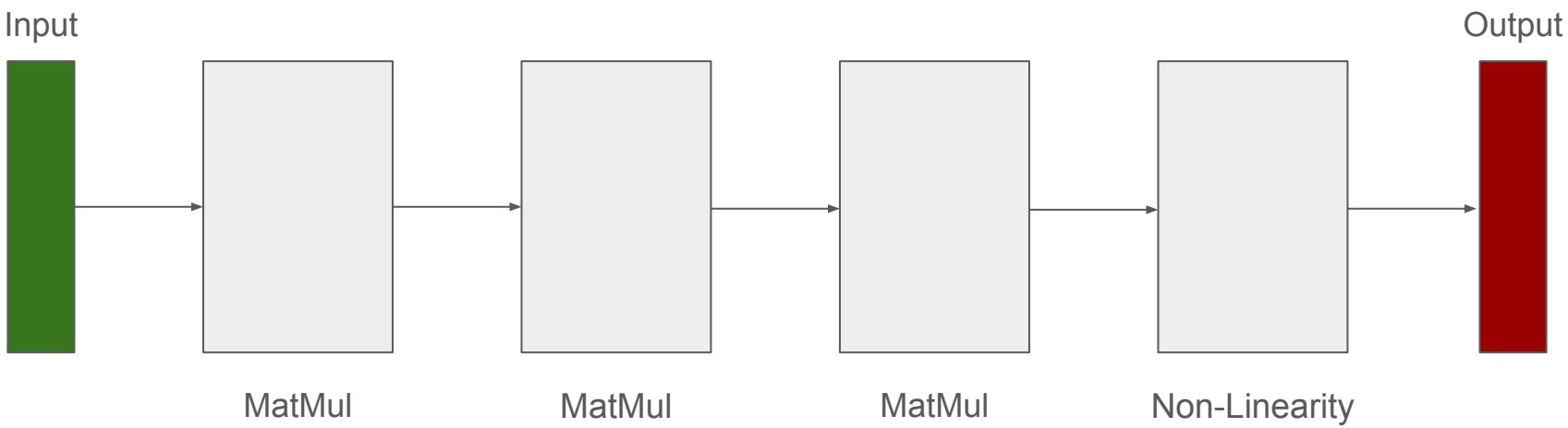
Tensor Parallelism

Data Parallelism

Parallelism in Machine Learning - Pipeline Parallelism

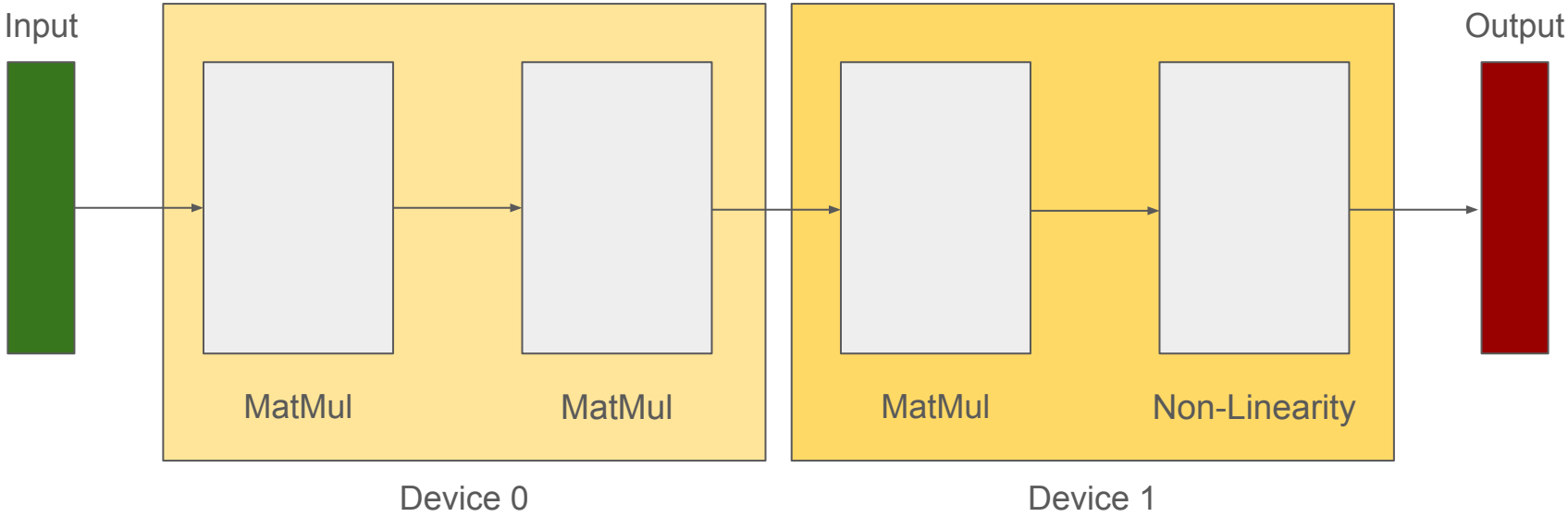
Split the model's forward/backward pass into p stages executed on p devices

- Communicate output of one stage to the next stage
- Uses point-to-point communication



Parallelism in Machine Learning - Pipeline Parallelism

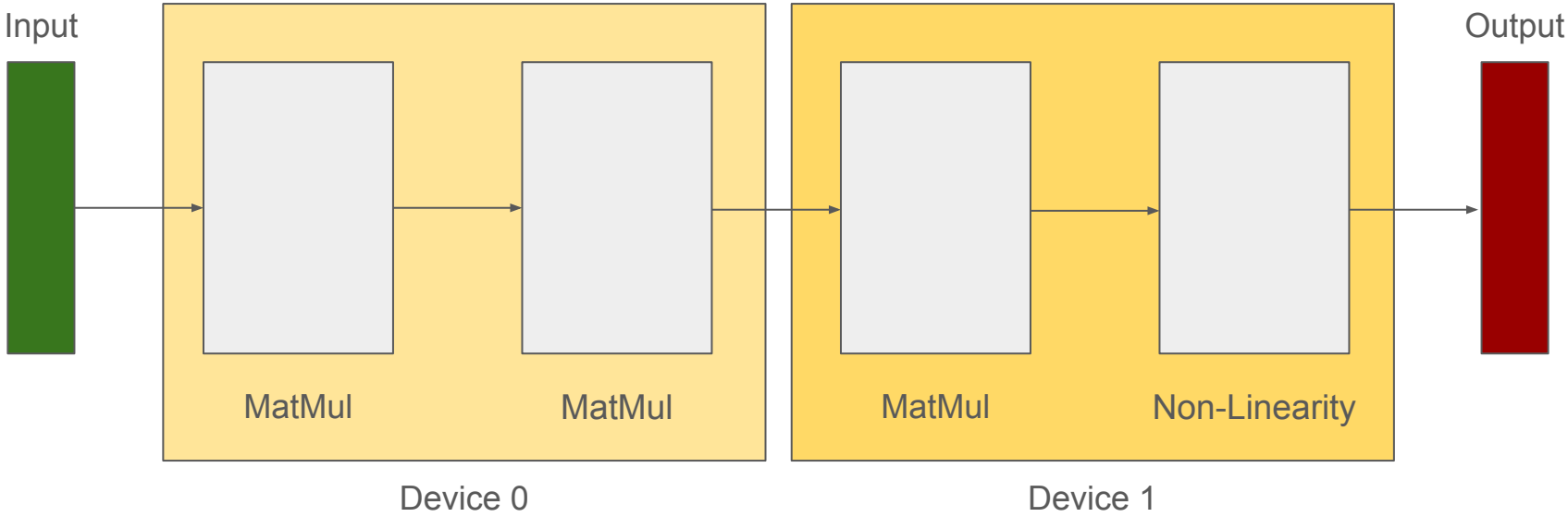
In the below example, the MLE decided to use `p = 2`, with the given partition



↑ Memory Efficiency, ↓ Communication Overhead, ↑ Latency

Parallelism in Machine Learning - Pipeline Parallelism

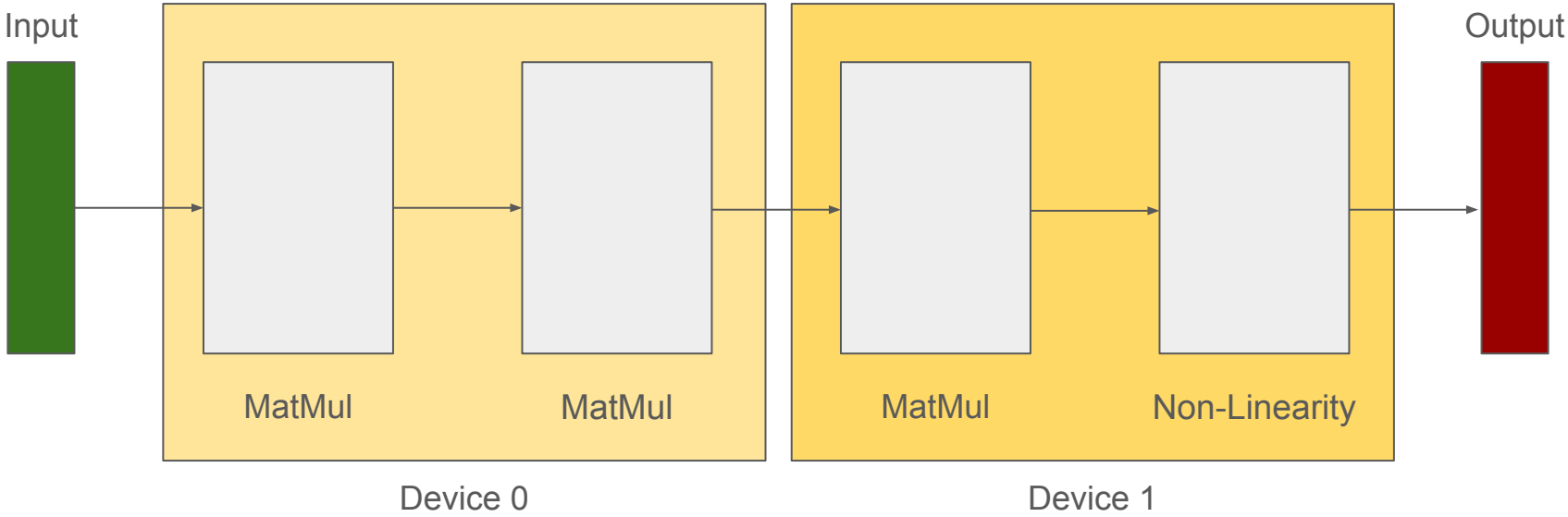
In the below example, the MLE decided to use `p = 2`, with the given partition



Training is hard, forward passes “get in the way” of the backward pass

Parallelism in Machine Learning - Pipeline Parallelism

In the below example, the MLE decided to use `p = 2`, with the given partition

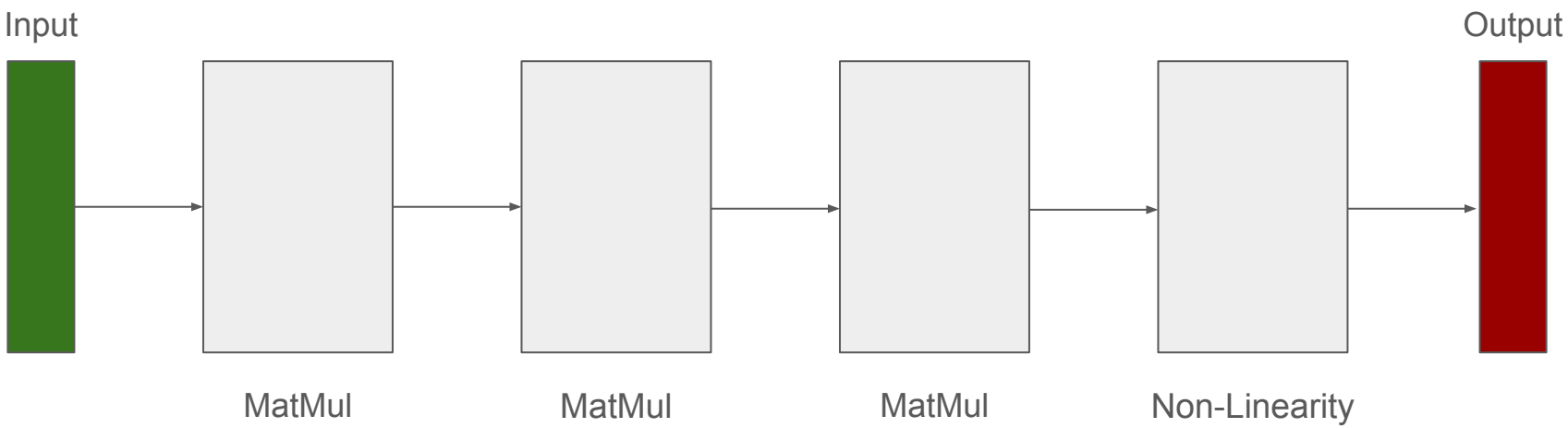


Placement is non-trivial, how do you know that `p`, alongside the partition is best?

Parallelism in Machine Learning - Tensor Parallelism

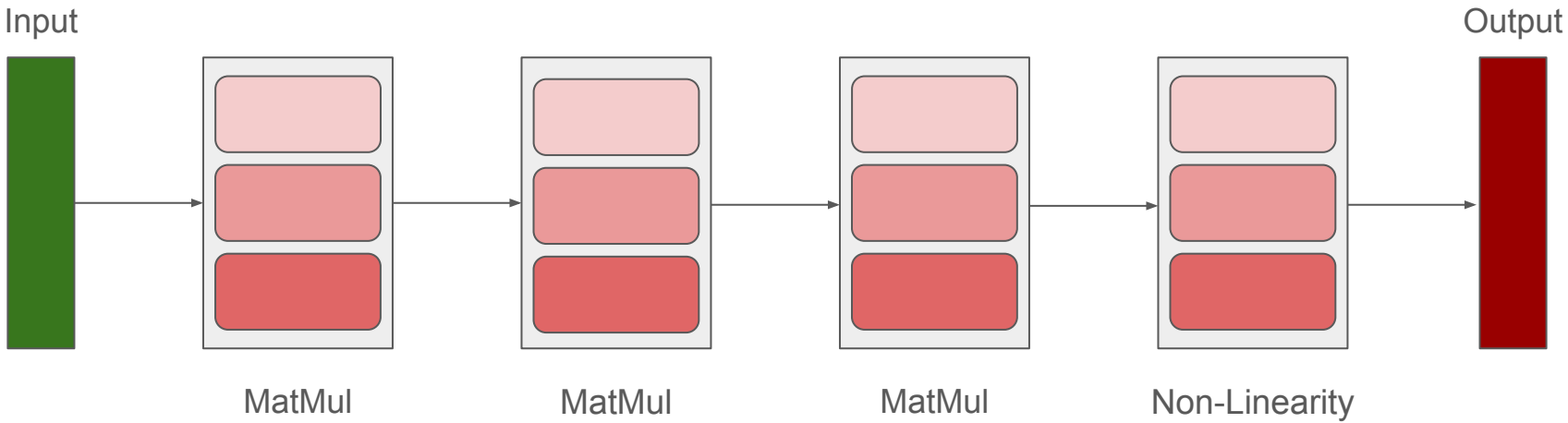
Split the models matrices into `t` “partial matrices” on `t` devices.

- Scatter inputs between all devices, gather outputs between all devices
- Uses collective communication



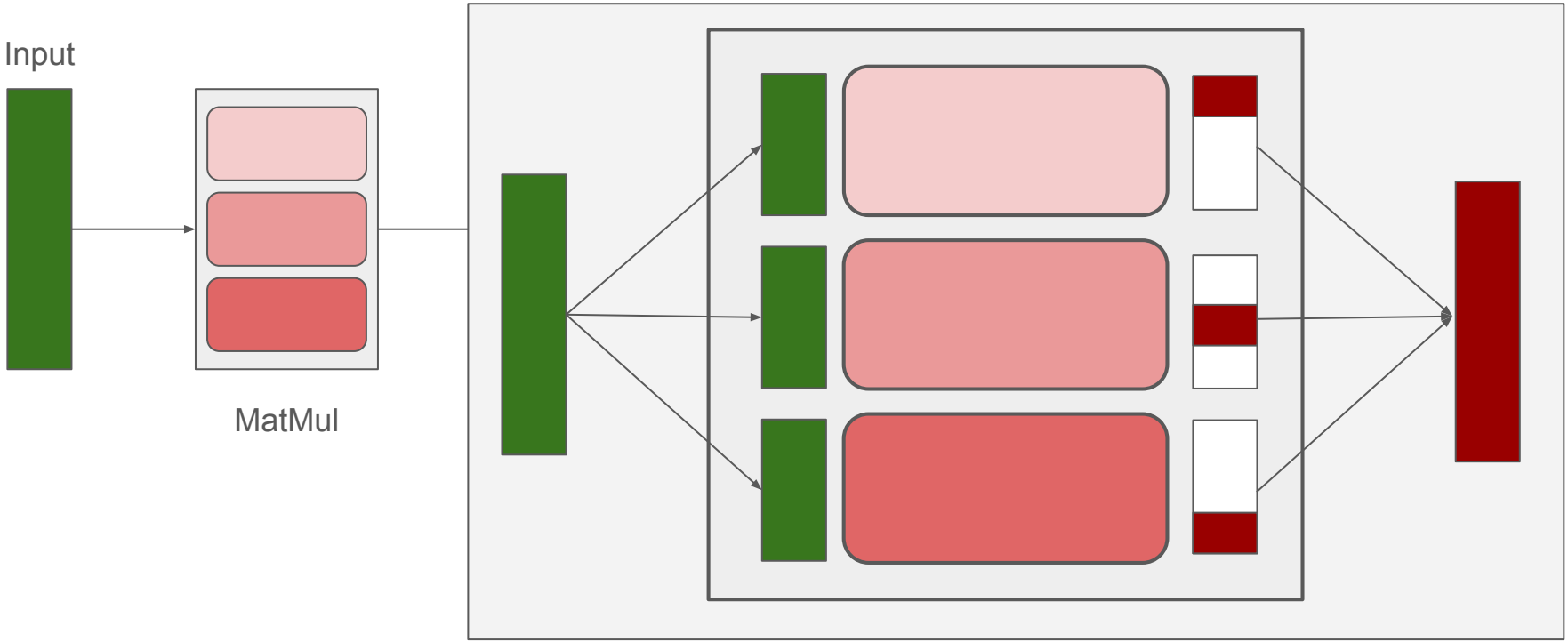
Parallelism in Machine Learning - Tensor Parallelism

In the below example, the MLE decided to use `t = 3`



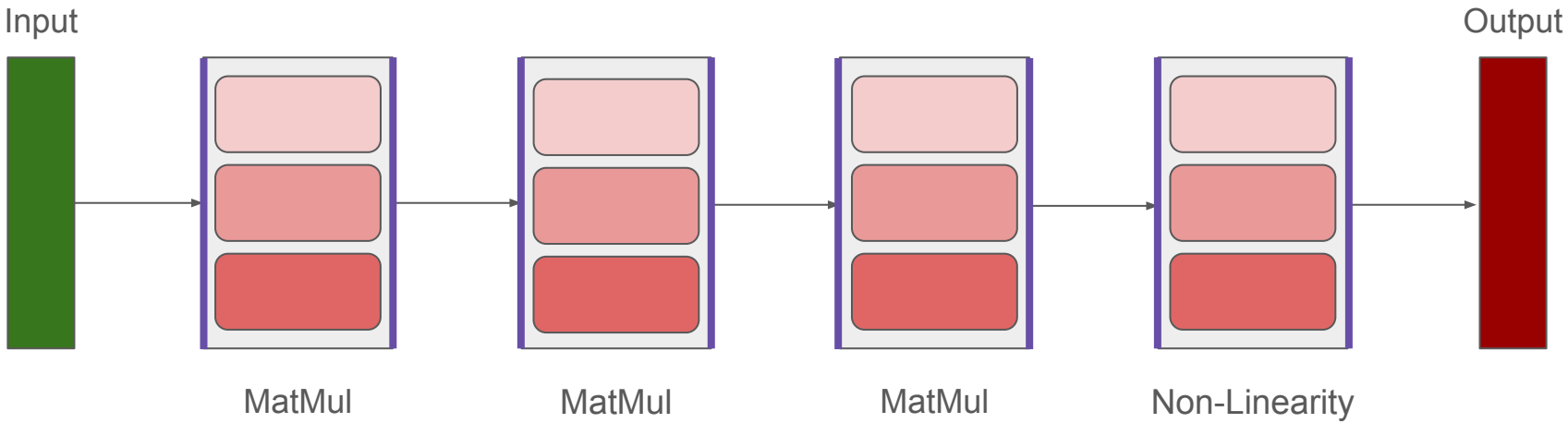
Parallelism in Machine Learning - Tensor Parallelism

In the below example, the MLE decided to use `t = 3`



Parallelism in Machine Learning - Tensor Parallelism

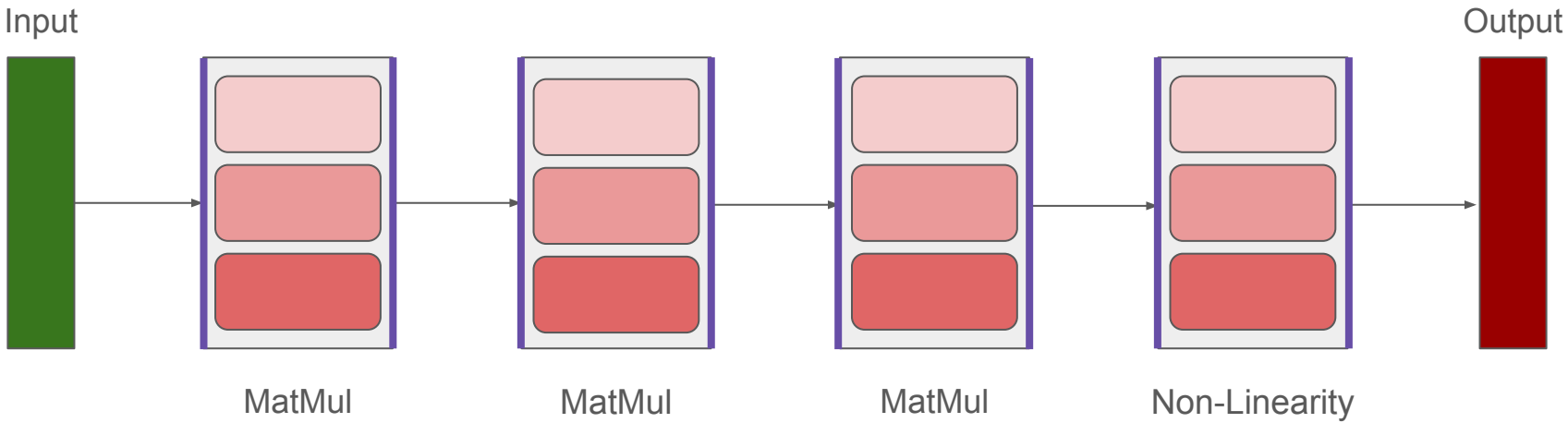
In the below example, the MLE decided to use `t = 3`



A lot of **collective communication** has to occur, but each device does less computation

Parallelism in Machine Learning - Tensor Parallelism

In the below example, the MLE decided to use `t = 3`

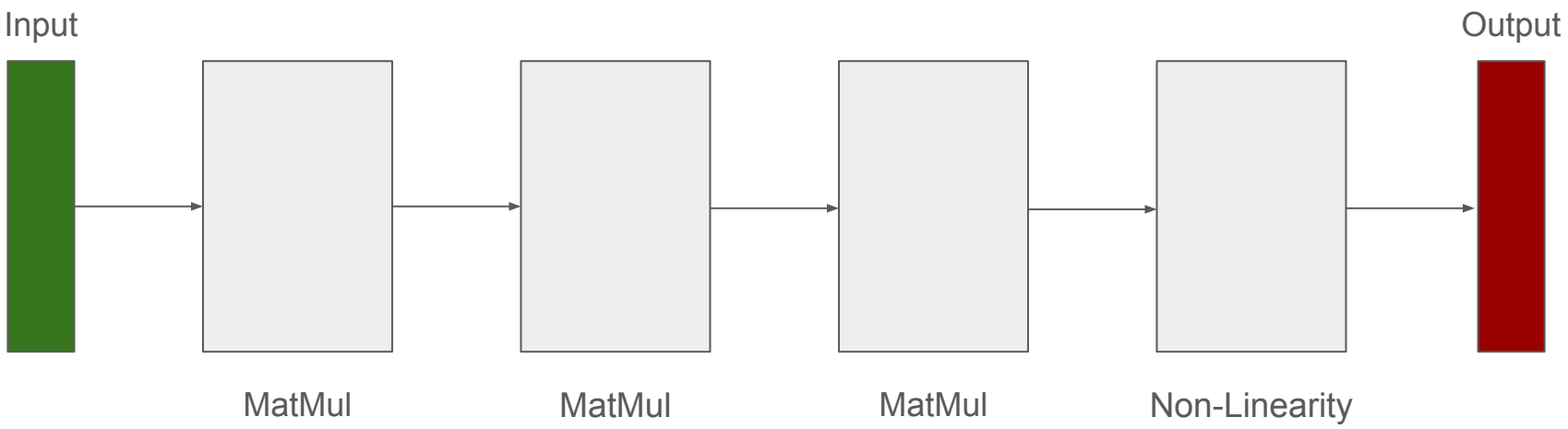


↑ Memory Efficiency, ↑ Communication Overhead, ↓ Latency

Parallelism in Machine Learning - Data Parallelism

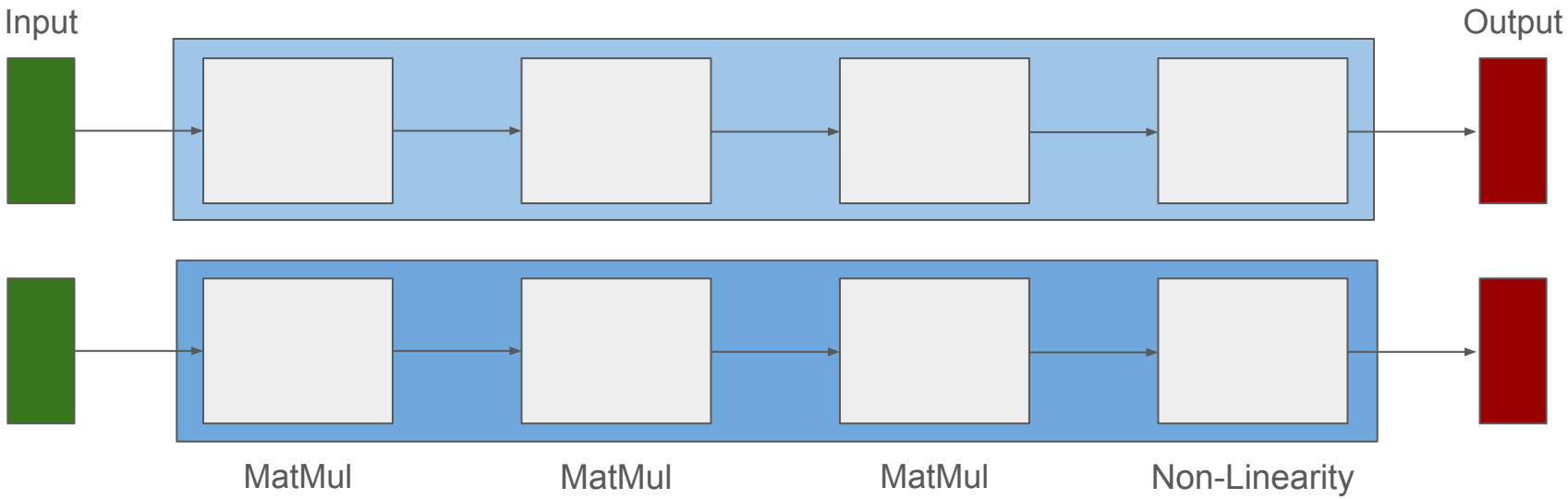
Replicate the model `d` times, each model placed on `d` devices to handle `d` streams

- Split apart inputs, gather outputs if necessary
- Uses collective communication



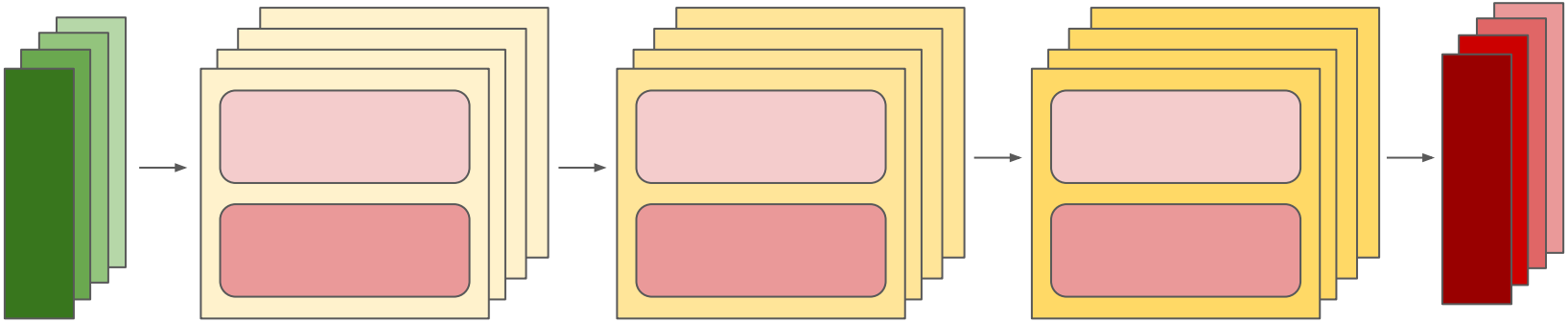
Parallelism in Machine Learning - Data Parallelism

In the below example, the MLE decided to use `d = 2`



↓ Memory Efficiency, ↓ Communication Overhead, ↑ Bandwidth

Production Parallelism in ML

The biggest models use all three forms of parallelism *composed* together

- Called 3D parallelism when combining these three
  - Generally, in a DP >> PP >> TP hierarchy, due to communication tradeoffs
  - Llama3 also used something called context parallelism → 4D parallelism
- Finding the optimal `(p, t, d)` plan is hard; an open area of research
  - Especially if you want to be able to recover from a crash after months of training…
  - RL makes this even worse, coordinating models with different memory requirements